Library build with task selection: first comparison and noise floor

Date: 2026-10-14 to 2026-10-15. One RTX 5090. Model: `claude-sonnet-5` for every writer and repair agent. Figures: `figs/` (make them with `make_figures.py`). Code: `scripts/taskselect/library_loop.py`.

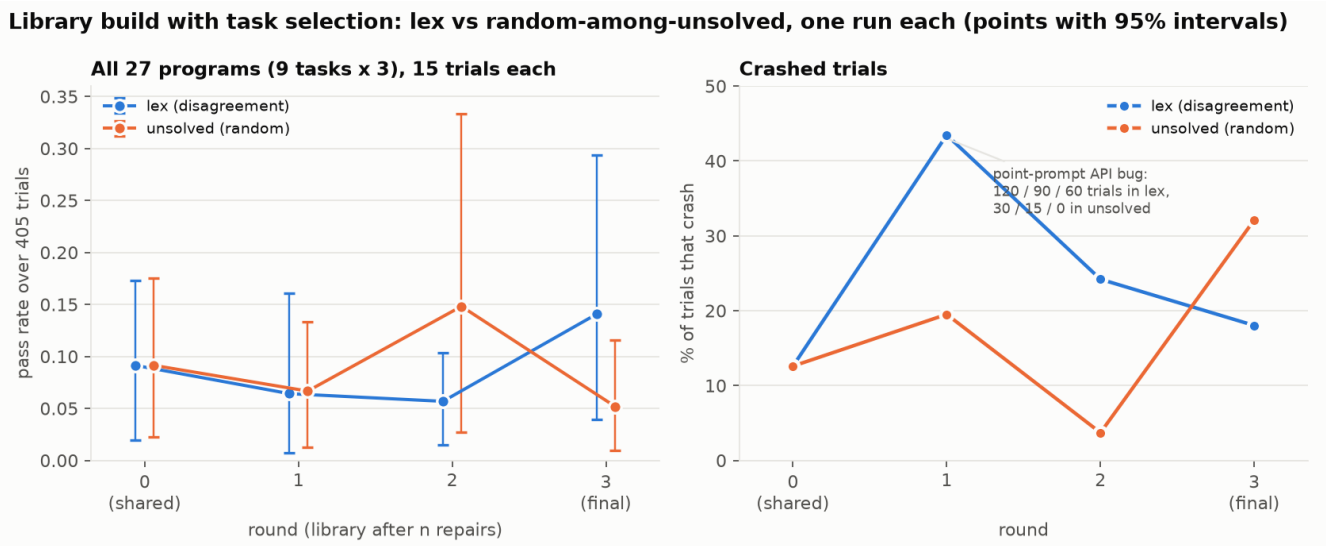
Summary

- We built a skill library from nothing in two ways, one run each: the next task to repair chosen by **disagreement** (`lex`), and chosen **at random among unsolved tasks** (`unsolved`).
- **No difference can be claimed.** Each round runs 27 programs (9 tasks x 3) for 15 trials each, 405 trials. The final pass rate is 0.141 (`lex`) against 0.052 (`unsolved`), but three independent generations of the *same* empty library already range from 0.035 to 0.091, and every 95% interval of the gap contains 0.
- Two clear library effects were seen, one in each arm, and both are real mechanisms, not selection effects: a perception rule that was used and helped, and a perception prompt that helped and then was not reused.
- A pipeline defect (point-prompt API misuse, never fixed in the writers' instructions) crashed up to 30% of the `lex` trials in rounds 1 to 3. It biases the comparison against `lex`.

Setup

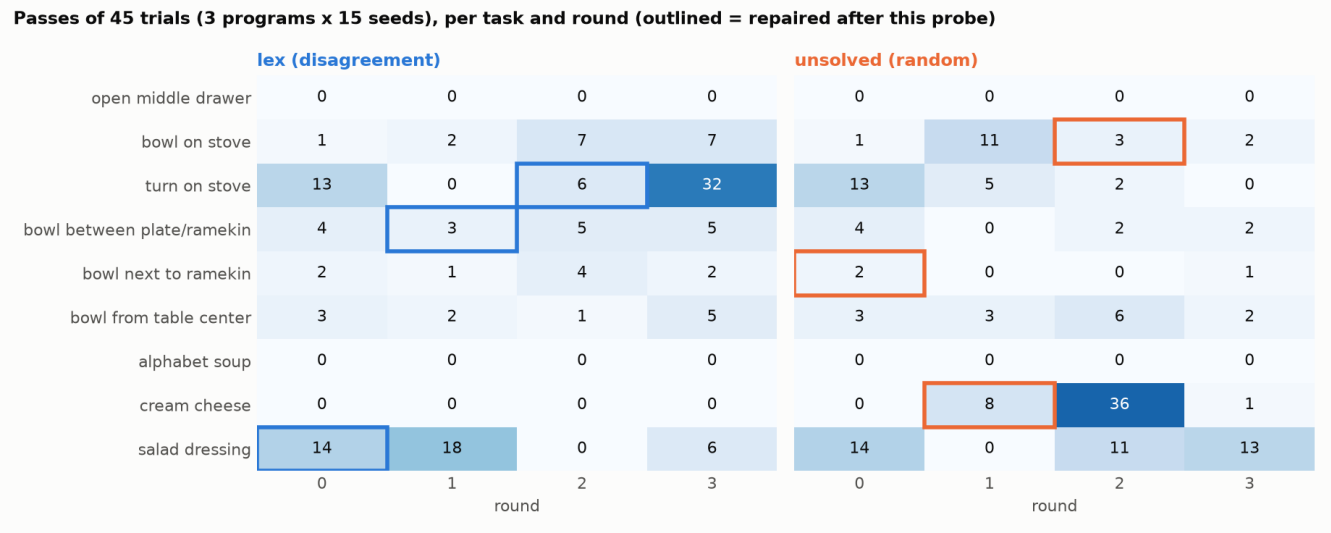
item	value
pool	9 swap tasks: goal (3), spatial (3), object (3), `library_9_swap`
start	empty skill library `L00`; round 0 shared by both arms (27 programs, 9 patched by hand for API errors, see Defects)
per round	3 fresh zero-shot programs per task from the current library, probed on seeds 51-65 (405 trials)
repair	one task per round, up to 5 iterations from its best program, then skill promotion
rounds	3 repairs, then a final probe (4 probes per arm)
selection	`lex`: highest per-seed split, then most tasks sharing the failure stage, then trace variety (explained, with a worked example, in `LEX.md`). `unsolved`: uniform among tasks with no program passing 15 of 15
differences between arms	`unsolved` probed with 4 workers, `lex` with 2 (speed only; outcomes are not deterministic at either)

1. The two arms



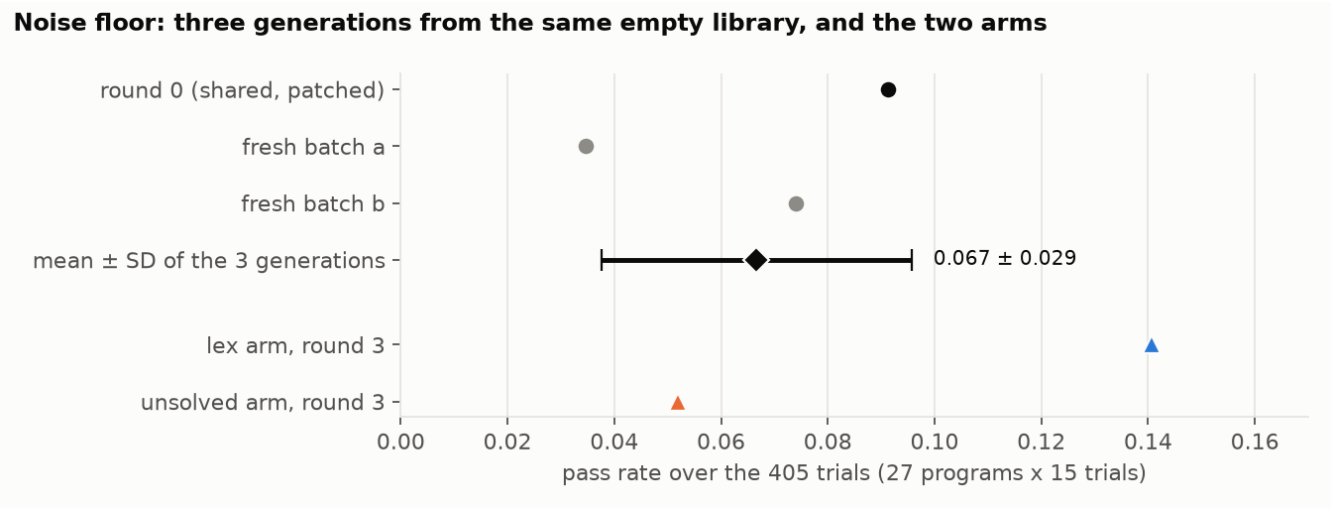
round	`lex` pass rate	`unsolved` pass rate	`lex` crashed trials	`unsolved` crashed trials
0 (shared)	0.091	0.091	13%	13%
1	0.064	0.067	43%	19%
2	0.057	0.148	24%	4%
3 (final)	0.141	0.052	18%	32%

- "Pass rate" is the passes of all 27 programs (9 tasks x 3 programs, 15 trials each) divided by the 405 trials.
- The best program per task is not used as a metric: a maximum over 3 noisy programs moves by 0.07 to 0.09 between identical generations.
- Solved tasks (one program passes 15 of 15): at most 1 of 9 in any round, in both arms.
- Repairs: `lex` picked salad dressing, bowl between plate and ramekin, turn on the stove. `unsolved` picked bowl next to the ramekin, cream cheese, bowl on the stove. Each arm solved one of its three repair tasks.
- The sign of the difference flips between rounds (+0.089 in round 3, -0.091 in round 2).



The per-task view (passes of 45 trials: 3 programs x 15 seeds) shows how unstable single tasks are: salad dressing in `lex` goes 14, 18, **0**, 6; cream cheese in `unsolved` goes 0, 8, **36**, 1. Neither can be explained by the library alone.

2. Noise floor



Three independent generations of programs from the same empty library, same pool, same seeds. Each generation is 27 programs (9 tasks x 3) with 15 trials each, 405 trials. The plot shows the pass rate over those trials, the mean and sample SD of the three generations, and the final value of each arm.

generation	pass rate over 405 trials	passes of 405	uncaught exceptions
round 0 (shared, 9 programs patched by hand)	0.091	37	51
fresh batch a	0.035	14	63
fresh batch b	0.074	30	0
mean ± SD of the 3 generations	0.067 ± 0.029		
`lex`, round 3	0.141		
`unsolved`, round 3	0.052		

- The SD across generations is 0.029 (pooled per-task estimate 0.030). This is the noise on one run.

- Both arms' final values lie within about 2.6 SD of the mean (`lex` +2.5 SD, `unsolved` -0.5 SD).
- The clean pair is a against b (same process, no patch): 0.035 against 0.074 (14 against 30 passes of 405).
- The round-0 generation is the highest, and it is the only one that was patched. So part of its gap to a and b is the patch.
- Execution noise adds to this: re-running identical programs on the same seeds changed 11 of 15 per-seed outcomes for one program (earlier control).

Reading the arms against this. The difference between two single runs has an SD of about 0.04.

round	gap `lex` - `unsolved`	95% interval (same tasks, seeds resampled)
1	-0.002	[-0.101, 0.121]
2	-0.091	[-0.291, 0.052]
3 (final)	+0.089	[-0.032, 0.269]

All three intervals contain 0. The final-round gap is about 2 noise SDs, but it appears in **both directions** (round 2 favours `unsolved` by the same amount). With one run per arm, three noise points and several metrics and rounds looked at, that cannot be read as an effect of the selection rule. It does show that library changes or generation variance move the metric by that much between rounds. A comparison needs several repetitions per arm, which is what the campaign now running provides.

3. The two libraries

	`lex` **(L03)**	`unsolved` **(L03)**
files	grasp, localize, manipulation, transport, README: 18 KB	localize, manipulation: 15 KB
edits	additions only, no rule removed or corrected	additions only
general rules	grasp plausibility filter; proprioceptive grasp check; min-thickness filter; landmark exclusion	out-of-vocabulary prompts are not "absent"; wrist-camera re-localisation with a displacement gate; floor-based drop height; learned planner then forced top-down
task-specific	push-flick for the stove knob; `DROP_CLEARANCE = 0.11`	gate values 0.08 and 0.12 m with the cream-cheese basket as evidence

Neither library is clearly better. `lex` covers more skill categories because its three repairs hit different failure kinds.

4. Which library elements measurably helped

element	evidence	verdict
min-thickness filter (`lex`, from the stove repair)	no round-1 or round-2 stove program implements a thickness filter (one comment mentions "flat"); all three round-3 programs do; passes 0/1/5 become 8/10/14	best evidence in the data
describe by appearance (`unsolved`, from the ramekin repair)	all 3 cream-cheese programs in rounds 1 and 2 use the prompt "white and blue box"; round 2 passes 11/15/10	helped, then lost: no round-3 program used it (1 pass of 45), with the rule still in the library
learned planner then forced top-down (`unsolved`, round 2)	programs calling `plan_grasp` rose from 13 to 19 of 27; "No grasp candidates found" 31 times, "All grasp attempts failed" 31 times; crashed trials 4% to 32%	correlation only; harmful in that round
drop clearance, yaw fallbacks	no durable effect	neutral

Why a better element appeared at all:

1. **The valuable elements are perception diagnoses**: why the object was not found or was wrongly picked. Control tweaks (clearances, yaw fallbacks, planner settings) gave variable results.
2. **A repair's apparent gain can be pure noise.** In the stove repair, iterations 2 and 5 are byte-identical programs (checked with `cmp`) and scored 7/15 and 13/15; the agent itself promoted a caution about this.
3. **Knowledge is not reliably reapplied.** A prompt that worked in two rounds was unused in the third with the library unchanged. The library stored the rule, not the concrete prompt; writers are not required to follow it.

5. Defects and deviations (read before trusting the numbers)

- **Point-prompt API misuse recurred in every later round.** The function takes one `(x, y)` pair; programs passed a list. The allowed-API list in `CLAUDE.md` says `points`, which invites it. I patched round 0 by hand and did not add the warning to the writers' instructions as intended. Trials with the resulting 422 error:

arm	round 1	round 2	round 3
`lex`	120	90	60
`unsolved`	30	15	0

- **Round 0 was patched by hand**: 9 programs (missing `import numpy as np`, list-wrapped point prompt, dict passed as a mask). The log is `outputs/library_*/round_00/patch_log.md`. Later rounds and the noise batches were not patched.
- **Molmo is off**, as in the upstream TC1 runs. It was removed from 11 round-0 programs and the writers were told not to use it.
- **Worker count** differed between the arms (2 against 4); a control showed it is not a source of bias.
- **One run per arm.** Nothing here is a significance statement.

Where things are, by experiment id

All paths are under `aspire/sim/outputs/` (gitignored). The live directory `.claude/libero/skills/` is only a working copy: each run saves its own libraries, so it is restored to the committed version when no run is active.

experiment id	what it is	libraries	probes and selections	repairs
`library_lex_r1`	arm `lex`, 3 repairs + final probe	`library/L00` (empty) to `L03`	`round_NN/probe.json`, `round_NN/selection.json`	`round_NN/fixloop/<suite>/<task>/findings.md`, `skill_promotions.jsonl`
`library_unsolved_r1`	arm `unsolved` (also holds the shared round 0)	`library/L00` to `L03`	same	same
`library_noise_a`, `library_noise_b`	fresh round-0 generations from the empty library	`library/L00`	`round_00/probe.json`	none
`skill_snapshots/family_start`	skills after the first single-task fix loop; equals the committed `.claude/libero/skills`	5 files	none	none

Each run also has `state.json` (pool, rounds, settings) and `coordinator.md` (the exact prompt it was started with). Round 0 of `library_lex_r1` is a byte-identical copy of the shared one (`probe.json` compared with `cmp`).

6. What this supports and what comes next

Supported: the pipeline runs end to end; a perception-level skill can raise a task's pass rate in later rounds; library edits can also lower it; run-to-run noise is as large as the differences seen.

Not supported: that disagreement selection beats random selection, in either direction.

Next, in order:

1. Add the point-prompt line and `import numpy as np` to the writers' instructions in every round, in both arms.
2. Repeat each arm at least three times, or raise the programs per task from 3 to 6 so the metric is less noisy.
3. Record the concrete perception prompt that worked in the library, not only the rule.